RSCTF TECHNICAL PRACTICE PAPER

HOW RSCTF SCORES ATTACK & DEFENSE: THE EPOCHBALANCED MODEL

Dimas Maulana
rsctf Project · Competition Platform
Implementation baseline: release commit containing this handbook · Manuscript version 1.6 · 14 July 2026
Policy label: EpochBalanced, the sole implemented A&D scoring formula

ABSTRACT

In an Attack & Defense (A&D) competition, every team attacks opponents' services while keeping its own service copies operational. An automated checker tests whether each service still works. rsctf's `EpochBalanced` *model measures three outcomes: offense from submitted opponent flags, pairwise defense from opponent-flag pairs that remain uncaptured, and service-level agreement (SLA) from checker results. An epoch is a fixed group of rounds. For each service in each epoch, the model combines 40% offense, 40% defense, and a 20% geometric balance term that rewards doing both. SLA then multiplies that complete result. A bounded scarcity term adds limited offense credit when few teams capture the same flag. Every complete epoch has equal weight; a shortened final epoch receives weight* `r/n`*, where* `r` *is its played-round count and* `n` *is the configured full length. Settled is the primary ranking value. Exact Settled ties use Live, offense, defense, SLA, and participation ID, in that order; unfinished evidence therefore affects ranking only through the Live tie-break. AI-assisted tools can accelerate exploit and patch work, but rsctf records outcomes rather than authorship. Appendix C maps the equations, timing, fault rules, and settlement process to the release commit that contains this handbook. Section 5 verifies the arithmetic with a deterministic five-team example. The repository contains no live-event behavioral dataset, so this report does not claim that the model causes sustained engagement, behavioral fairness, or any other player behavior.*

Keywords: *attack-defense CTF; cybersecurity competition; epoch scoring; service-level agreement; adversarial robustness; human-AI teaming; competitive fairness; rsctf*

Document status: versioned technical-practice report, not a claim of peer review. Implementation claims are traceable to Appendix C.

START HERE: A&D IN 60 SECONDS

The big idea

Keep your service working while stealing flags from other teams. For each challenge, every team has its own copy of the same service. You protect your copy and attack the copies assigned to you.

Play in four steps

1. **Protect yours:** fix the weakness without breaking the service.
2. **Attack theirs:** use that weakness against teams on your target list.
3. **Submit stolen flags:** send each flag to RSCTF before it expires.
4. **Repeat:** new flags appear in the next round.

Protecting and attacking happen at the same time. Do not wait for one job to finish before starting the other.

How points work

You earn more by doing three things:

- stealing and submitting opponents' flags;
- stopping opponents from stealing yours; and
- keeping your service working when RSCTF checks it.

The highest scores require all three. If your service often fails its checks, your whole score drops even when your attacks succeed.

What starts over

New short-lived flags appear every round. Submit them quickly. Your service copy normally keeps running between rounds, so your patches stay in place.

Read the board

Settled is the primary ranking value. **Live** can still change and breaks an exact Settled tie before offense, defense, SLA, and participation ID.

New player? Continue with Sections 2.1 and 3, then read the scoreboard guide in Section 5.3 and the checklist in Section 7. The later sections explain the exact mathematics and organizer controls.

Key terms used in this handbook

- A **service copy** is your team's running instance of a vulnerable challenge.
- A **flag** is a short-lived secret value. Your team earns attack credit when it submits an opponent's valid flag before the flag expires.
- A **target** is an opponent service that rsctf has assigned your team to attack.
- A **checker** is the event program that tests whether a service responds correctly and handles the current flag.

- A **round**, also called a **tick**, is one cycle of new flag records, checks, attacks, and submissions.
- An **epoch** is a fixed group of rounds that rsctf scores as one result.
- **SLA** is the service reliability rate calculated from checker evidence after the fault rules in Section 4.5 are applied.

Figure 1. A&D in one picture. Every team protects its own service copy while attacking assigned opponents. RSCTF checks whether services work and accepts stolen flags. New flags begin the next round.

1. INTRODUCTION

A&D gives every accepted team an identical copy of each enabled vulnerable service. Four tasks run at the same time: keep your copies healthy, patch them without breaking the checker contract, exploit assigned opponent copies, and submit captured flags before they expire. A **checker contract** is the behavior that the event's checker expects, including network responses and current-flag handling.

Bring Your Own Container (BYOC) is rsctf's self-hosted mode. Your team runs the service container and relay agent on infrastructure that it controls. At the start of every round, rsctf creates the authoritative flag records. For BYOC, the platform attempts to deliver the new flag through the relay. For a managed container that is already running, rsctf records the new flag but does not inject it into the service. In both modes, rsctf gives the recorded flag to the custom checker. The platform accepts a submitted flag only when it belongs to one of your assigned targets and is still within its configured lifetime [1].

The scoring rule combines 40% offense, 40% pairwise defense, and a 20% balance term. SLA multiplies the result for that service. Every complete epoch has the same coefficient; a shortened final epoch receives a fraction based on the number of rounds played. Teams keep their exploit code and do not declare their patches to rsctf. The records therefore establish accepted flags and checker results. They do not establish who wrote an exploit, whether an opponent tried to attack, whether a particular patch stopped an attack, or whether a human or AI performed the work [1].

1.1 Design motivation under AI-assisted play

Teams can use AI to find vulnerabilities, draft exploits, or propose patches. This report contains no event data that measures how AI use changes live A&D performance. The scoring rule nevertheless creates three observable risks. A generated patch lowers SLA if it breaks behavior required by the checker. A slow or noisy exploit can miss the flag's active window. Offense-only automation loses balance credit as defense falls; the geometric term is zero when either offense or defense is zero. Event data would be required to test whether these incentives improve validation, rollback discipline, or sustained attack coverage.

Debono (2026) argues that binary Jeopardy scoring can converge on a common ceiling and calls for more granular measurement of solution performance [2]. EpochBalanced applies this principle to evidence that the platform can verify every round: accepted flags and checker results. Teams do not upload exploits or declare patches.

Bounded scarcity is limited extra offense credit for a flag captured by few teams. The cap prevents one early, rare capture from becoming an unlimited jackpot. Every complete epoch has weight `1`, although one later epoch moves the cumulative

average less after many epochs have already been scored. Because SLA multiplies the local score, a patch that breaks the checker lowers the result.

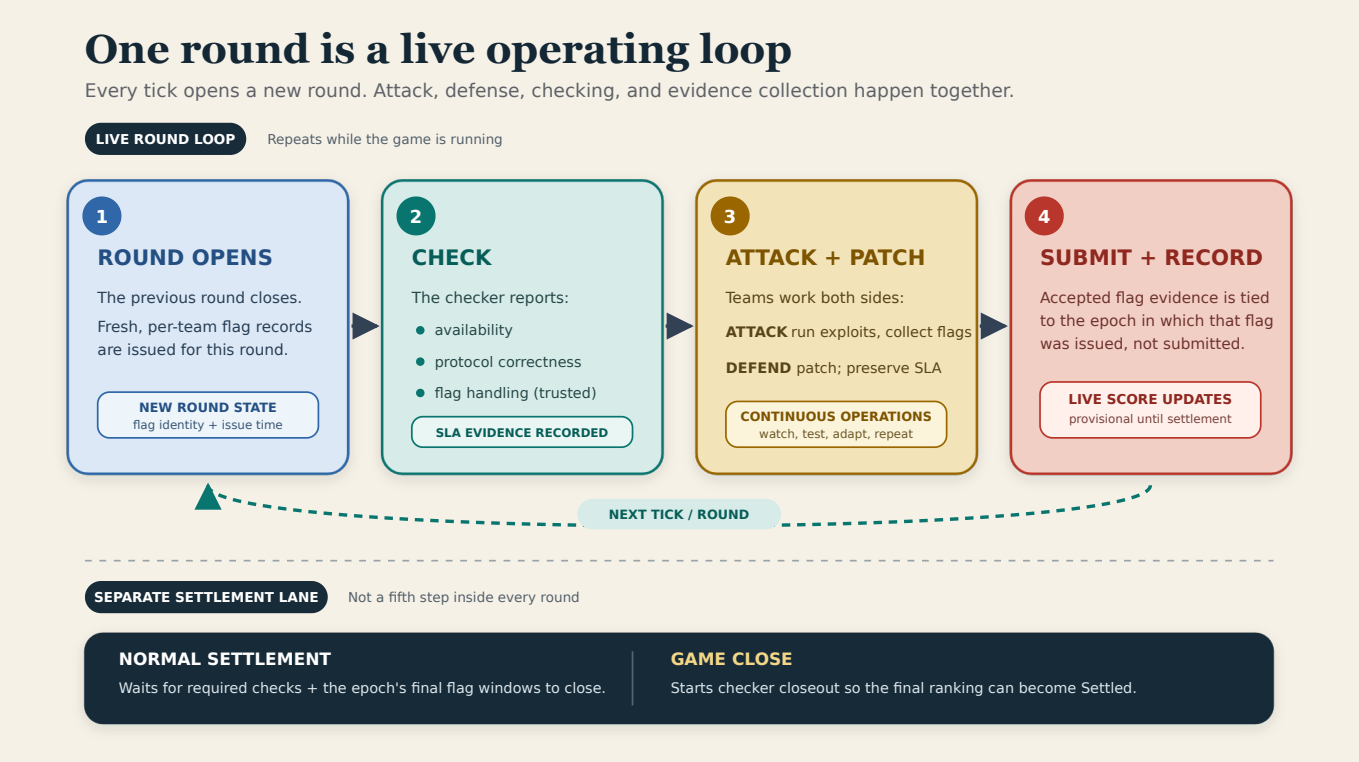

Figure 2. Live A&D round and asynchronous settlement workflow. The four operational steps repeat each tick; settlement is not a fifth step inside the round.

1.2 Relation to established A&D platforms

FAUST CTF 2025 provides a useful numerical comparison because it publishes both its timing and scoring rules. It uses three-minute ticks, issues one flag for every team-service pair in every tick, and accepts each flag for five ticks. Its score adds offense, defense, and SLA for each service. Offense combines the capture count with extra credit based on how few teams captured a flag. Defense subtracts a sublinear function of the number of distinct teams that captured an owned flag. SLA adds credit for `Up` and `Recovering` ticks [3]. EpochBalanced uses the same broad evidence types, including flags and checker results, but chooses bounded rarity, positive pairwise-defense normalization, multiplicative local SLA, and bounded epoch aggregation. The comparison identifies policy differences; it does not show that either model is superior.

The public FAUST CTF Gameserver separates its work among several deployable components. A controller creates the flag records for each tick. Checker scripts place and retrieve flags. Submission servers record captures. During the next tick, the system calculates the previous tick's score. PostgreSQL coordinates these components [4]. rsctf includes more of the container, VPN, and BYOC lifecycle in one platform, but it does not yet inject each new flag into an already-running managed container [1]. The two systems can therefore be compared as architectures, but their network interfaces are not compatible.

iCTF 2021 distributes points in a different way. Each round has a fixed pool of `50 x teams x services` points. An available service that nobody exploits keeps its allocation. If attackers exploit an available service, they share that allocation. If a service is down, teams whose corresponding service remains up share its allocation [5]. This constant-sum transfer differs from FAUST CTF's accumulating components and EpochBalanced's bounded weighted average. All three systems score repeated operational outcomes, but each system encourages different behavior through its allocation rule.

2. SYSTEM MODEL AND COMPETITION PROTOCOL

2.1 Round, flag window, and epoch

Table 1. Operational clocks and their scoring consequences.

Term	Meaning	Why it matters
Tick / round	One operational cycle in which flags, checks, attacks, and service state are recorded.	This is the smallest unit of live play.
Flag window	The number of ticks for which an issued flag remains acceptable.	A flag can remain valid after its originating tick. The default engine window is five ticks, but event settings may differ.
Epoch	A fixed group of ticks scored as one result.	A complete epoch has weight `1`. A partial final epoch with `r` played rounds out of `n` configured rounds has weight `r/n`. The epoch becomes final only after its checks and last flag windows close.

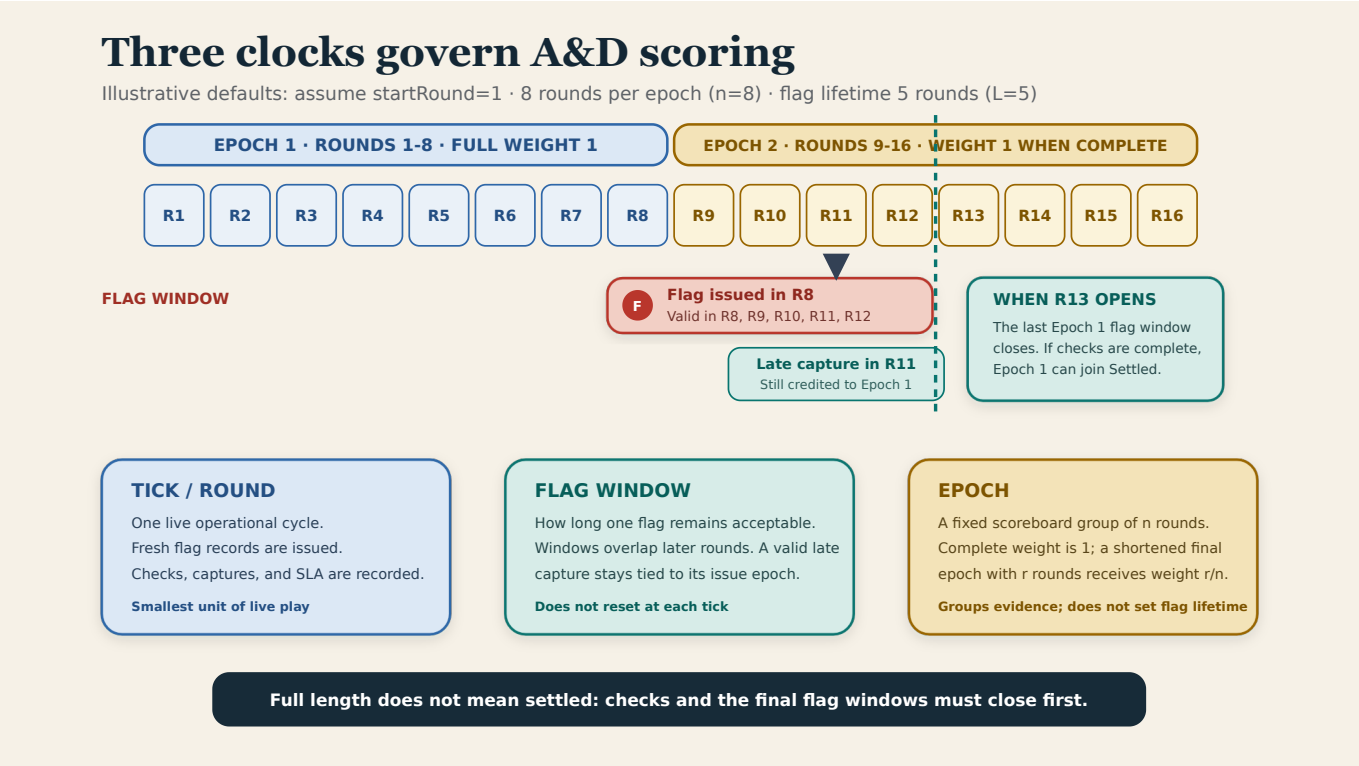

Figure 3. Rounds, flag windows, and epochs for the illustrative values `startRound=1`, `n=8`, and `L=5`. The round-8 flag remains valid through round 12 and retains Epoch 1 attribution.

Figure 3 uses `n=8` rounds per epoch and a flag lifetime of `L=5` rounds as an example; organizers can choose other permitted values. A flag issued in round `N` remains valid through round `N+L-1`. If your team submits it later, rsctf still assigns the capture to the epoch in which the flag was issued.

2.2 Competition lifecycle

An rsctf A&D event has six operational phases. Some phases overlap. For example, teams can play a new round while rsctf prepares an earlier epoch for settlement. The list describes what the platform is doing, not six exclusive database states.

1. **Provisioning and warmup:** organizers register one working instance of every enabled service for every accepted team. rsctf can manage the instance, or the team can host it through BYOC. Organizers also prepare custom checkers that understand the current flag. Teams use warmup time to connect and test their services. The platform can create operational rounds before ranked scoring begins.

2. **Scoring start:** rsctf records one global `startRound`, meaning the first ranked round. It does so only after three conditions hold: the game has at least two accepted teams; every accepted team has a database row for every enabled service; and every enabled A&D challenge has prepared checker files. This automatic gate does not test whether services are reachable, so organizers must perform that test during warmup. Evidence from earlier rounds does not count.
3. **Live rounds:** rsctf opens a round, creates new flag records, runs checkers, and publishes targets. Teams attack, patch, and submit flags. Accepted submissions become durable database evidence.
4. **Epoch settlement:** fixed groups of rounds feed the Live score. A complete group can remain Live because flags from its final rounds can still be valid. The epoch enters Settled only after its checker work and submission windows close.
5. **Event close:** the published deadline immediately stops flag submissions and prevents new normal rounds. A checker pass that started before the deadline can finish during closeout. After the grace threshold, rsctf can apply its fallback rule to results that are still missing.
6. **Fully settled:** rsctf seals the final round, resolves missing checker evidence under the fault rules, and stores a score for every ranked epoch, including a shortened final epoch. Live then equals Settled. Awards should wait for this state.

2.2.1 Round timing is scheduled, not a perfect metronome

The configured tick duration is the intended round length. A background scheduler repeatedly finds games that are due, prepares their rounds, checks that service records match the event configuration, publishes flags, and runs checker work. Heavy load can delay the exact opening time. Player automation should read `currentRound` and `roundEndsAt` from the API instead of assuming that every round starts on an exact wall-clock second [1].

Such a delay does not change epoch weight. rsctf counts scoring rounds that it successfully opened and stored; it does not count elapsed minutes.

2.3 Per-round state transition

1. **The platform prepares the round.** rsctf creates the official rotating-flag records and publishes each team's assigned targets. For BYOC, it attempts to deliver the flag through the relay's shared file. For a managed container that is already running, it records the flag but does not inject the value into the service.
2. **The checker tests every service.** rsctf sets `RSCTF_ACTION=check`, provides the recorded flag as `RSCTF_FLAG`, and exports compatibility aliases for older checker images. rsctf trusts the checker's exit status. An `Ok` exit marks the flag as verified. The platform does not compare the flag a second time and does not run a separate flag-installation action.
3. **Teams attack and patch.** Offensive tools target assigned opponent services while defenders harden their own service copies.
4. **rsctf validates submissions.** Each submitted flag must match the assigned victim and service, and its active lifetime must not have ended.
5. **The Live score updates.** Accepted flags, eligible rotating flags, and checker results enter the current epoch calculation. Settled changes only when the whole epoch can become final; ending one round is not sufficient [1].

Checker results, accepted flags, flag eligibility, and target assignments are authoritative platform records. No accepted capture means that rsctf observed no successful submission for that attacker-flag pair. It does not mean that the attacker tried, nor does it explain why no capture occurred.

3. PLAYER OPERATIONS

3.1 Open the A&D toolkit

The game page provides a toolkit for your team. Depending on the event, it includes:

- A WireGuard configuration
- SSH public-key management and a bastion address
- Your current target list
- A team-shared Bearer token for automated submissions. The token belongs to one accepted team entry, called a **participation**, in one game.
- Batch flag submission
- Service state, current tick, and official epoch scoreboard details
- Bring Your Own Container (BYOC) relay-agent setup and generated Docker Compose configuration for self-hosted challenges

3.2 Connect to the VPN

1. Download the WireGuard configuration from the game.
2. Store it privately; it grants your team network access.
3. Import it into the official WireGuard client or a compatible client.
4. Activate the tunnel.
5. Test one address from the current target list.

Only routes configured by the organizer should traverse the tunnel. If enabling it breaks unrelated Internet access, disconnect and report the behavior to the organizer.

3.3 Access your service

When SSH is enabled, add or generate the key shown by the toolkit and use the exact command displayed by rsctf. The username normally identifies the challenge, while your registered key identifies your participation.

Keep a backup and a repeatable patch script outside the running container. A reset can discard changes made only inside the instance.

3.4 Defend without breaking the service

- Verify that every service is reachable before the first scored tick.

- Preserve the expected protocol and flag/checker behavior while patching.
- Watch availability or SLA results after every change.
- Make one controlled change at a time and keep a clean rollback path.
- Do not block the checker, BYOC flag relay, or required game traffic.
- Treat a checker failure as an operational incident, not as proof that a patch stopped an exploit.

A perfect security patch that breaks the required service contract is still a failed service. Availability multiplies the official service score.

3.5 Attack and submit accepted flags

Attack only addresses listed as targets for the current game. Submit stolen flags through the UI or the documented A&D endpoint. A batch contains from 1 to 100 flags.

The Bearer token is shared by the whole participation and scoped to this game. Only the member who rotates it receives the new plaintext in their browser, but that rotation replaces the token used by every teammate. Coordinate rotations, redistribute it only through a secure team channel, and update every submission worker together. Never commit it to a repository, paste it in a writeup, or share it outside your team.

rsctf accepts a flag when it matches one of your assigned targets and arrives during its active window. The resulting record proves that the authenticated participation submitted the right target's flag on time. It does not record how the team obtained the flag, who operated the exploit, which network path it used, or who wrote the exploit.

When a flag has at least four eligible opponents, a capture by few teams can receive a bounded scarcity adjustment. Here, **capturer count** means the number of distinct frozen-roster teams that submitted that flag. A low count is only a proxy for difficulty; it does not prove that the owner had installed a patch or that an attacker bypassed one.

3.5.1 Flag lifetime and attribution

Let `L` denote the event's flag lifetime in rounds. A flag issued in round `N` is valid from round `N` through round `N+L-1`. For example, with the common default `L=5`, a flag issued in round 10 remains valid in rounds 10, 11, 12, 13, and 14. It expires when round 15 opens.

Flags from several rounds can therefore remain valid at the same time. Submit quickly, but do not discard a captured flag merely because the scoreboard has advanced by one round. rsctf assigns an accepted capture to the epoch in which the flag was issued, not the epoch in which your team submitted it. An epoch can therefore look complete while remaining Live until its last flag windows close.

The game deadline overrides remaining lifetime: submissions close immediately when the event ends. Self-attacks are rejected, and the same attacker can score the same flag only once [1].

3.5.2 Scripted submission

The in-game toolkit shows the exact base URL and lets a team member rotate the participation's shared, game-scoped token. A minimal batch submit looks like this:

```
AD_TOKEN='replace-with-team-token'
curl -X POST 'https://ctf.example/api/Game/GAME_ID/Ad/Submit' \
  -H "Authorization: Bearer ${AD_TOKEN}" \
  -H 'Content-Type: application/json' \
  -d '{"flags":["flag{captured_from_team_b}","flag{another_capture}"]}'
```

The endpoint accepts 1 to 100 flags per request. Each result reports one of the following states:

Table 2. Batch submission decisions and recommended player actions.

Status	Meaning	Player action
`accepted`	Valid opponent flag inside its lifetime; evidence was recorded.	Keep the success and continue scanning targets.
`duplicate`	Your participation already submitted this flag.	Deduplicate locally; do not retry it.
`expired`	The flag was real but its valid round window closed.	Reduce exploit and submission latency.
`wrong`	The value is not a recognized game flag.	Check parsing, target selection, and stale output.
`self_attack`	The flag belongs to your own service.	Fix target filtering.

Status	Meaning	Player action
`not_started`, `paused`, or `ended`	Scoring is not currently accepting evidence.	Stop automated retries and follow organizer notices.

Batching reduces overhead, but do not wait so long that short-lived flags expire. Use the current target endpoint as the source of truth and respect its challenge, IP, port, and health fields.

4. EVIDENCE AND SCORING METHOD

The scorer reads three kinds of database evidence:

1. an **accepted flag submission**, which records a successful target-bound submission;
2. an **eligible rotating flag**, which is a flag allowed to enter an offense or defense denominator under the rules below; and
3. a **checker result**, which records service behavior for one round.

Teams keep their offensive tools. rsctf neither receives nor executes their exploit code. Appendix C identifies the exact source revision used to audit these implementation claims [1].

Official scoring starts on the first round that meets three database conditions: at least two teams are accepted; every accepted team has a row for every enabled A&D service; and every enabled A&D challenge has prepared custom checker files. This gate does not test service health. An `Offline` service or a placeholder without a container therefore does not delay scoring. The qualifying round becomes the global `startRound`.

At `startRound`, rsctf freezes the ranked roster of teams and services from the flag records. Every epoch reuses this snapshot. A team, service, or capture that falls outside the snapshot does not enter the official score. When a flag qualifies as an offense opportunity, every frozen opponent except the flag's owner receives that same opportunity. A checker failure therefore cannot give one attacker a smaller denominator than another [1].

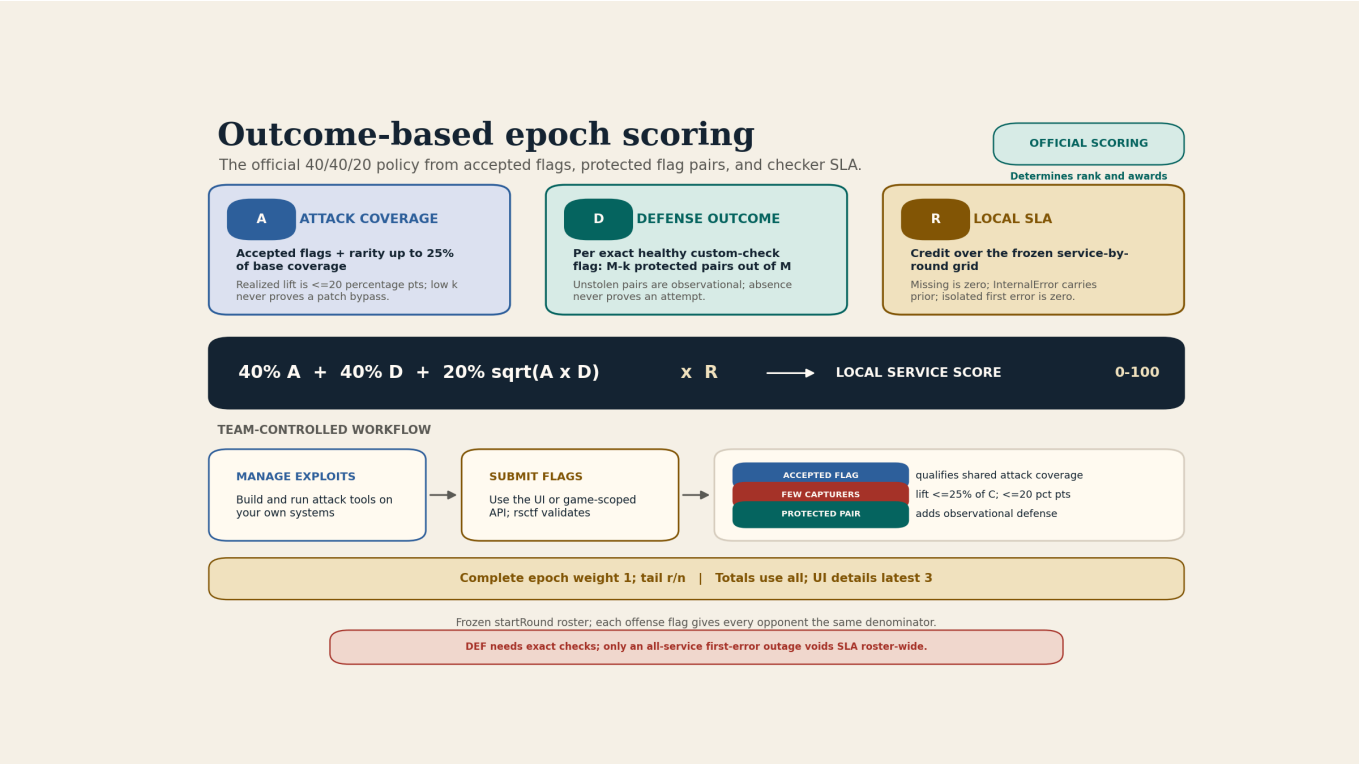

Figure 4. EpochBalanced evidence and aggregation pipeline. Accepted captures determine offense, eligible flag-aware checker outcomes determine pairwise defense, and local checker evidence determines SLA.

Diagram scope: the field-wide first-error outage rule covers every frozen service for one challenge in one round. It does not automatically cover other challenges in the event.

4.1 Team-controlled exploit workflow

1. Build, test, and manage exploit tools on your team's own systems.
2. Run them against only the current targets published by the game.
3. Submit captured flags through the UI or normal game-scoped submission API.
4. Let rsctf validate each flag and attribute accepted evidence to the epoch where that flag was issued.

4.2 Evidence interpretation

Let `M` be the number of eligible opponents for a flag, excluding the flag owner, and let `k` be the number of distinct eligible teams that submitted that flag. The **frozen roster** is the

team-service snapshot recorded at `startRound`.

The interpretation remains narrow. A rare capture has a low capturer count; it does not prove that the attacker bypassed a patch. An uncaptured flag raises the observed defense fraction; it does not prove that another team attempted an exploit or that a patch stopped one.

Defense counts attacker-flag pairs instead of marking the whole flag as either protected or lost. In a 20-team event, a flag owner has `M=19` eligible opponents. If one opponent captures the flag, the owner retains `18/19` defense opportunities for that flag.

Defense eligibility has a stricter evidence rule than offense. The flag needs an `Ok` result from a prepared, flag-aware custom checker. rsctf treats that result as confirmation of the current flag. A generic TCP reachability probe creates no defense opportunities. An accepted capture can place the flag in the shared attack denominator, but the capture alone cannot create defense credit.

Even a captured flag creates no defense denominator without an `Ok`, verified checker result. If a team has no checker-qualified own flags for one service in one epoch, Equation (2) has a zero denominator. rsctf then assigns `D=0`; it does not assign a neutral defense value.

Table 3. Mapping from observed platform records to scoring evidence.

Platform outcome	Official interpretation
A valid target-bound flag is accepted	The submission counts toward the authenticated team's attack coverage. It can show offense reachability even when that flag's checker result is not healthy.
An accepted flag has `M >= 4` eligible opponents and `k` distinct capturers	The flag contributes `(M-k)/M` to the scarcity sum. Across one epoch, `0.25H` cannot exceed 25% of base coverage `C`; clamping limits the absolute increase in `A` to `0.20`.
A healthy eligible rotating flag has `M` opponents and `k` distinct capturers	The owner protects `M-k` of `M` attacker-flag pairs. Each distinct capturer removes one pair. With only two teams, that single pair is the complete denominator.
A checker records service credit	The credit enters the local reliability, or SLA, rate `R`.
A prepared flag-aware custom checker receives the current flag and exits `Ok`	rsctf marks the flag verified, which allows it to create defense opportunities. rsctf trusts the checker contract. A fallback TCP probe cannot qualify a flag for defense.

4.3 Official service formula

The platform calculates one result for each team, service, and epoch. The notation below fixes team t, service s, and epoch e:

- An **attack opportunity** is an opponent flag from the frozen roster that either received an `Ok`, flag-verified custom-checker result or was captured by at least one frozen-roster attacker.
- N_{att} is the number of attack opportunities.
- F_{acc} is the set of distinct opportunities for which team t submitted an accepted capture.
- F_{def} is the set of team t's own flags that received an `Ok`, flag-verified result.
- For flag f, M_f is its frozen eligible-opponent count and k_f is its distinct frozen-roster capturer count.

The scorer first calculates accepted capture coverage C and normalized scarcity H:

$$C = \frac{|F_{\text{acc}}|}{N_{\text{att}}},$$

$$H = \frac{1}{N_{\text{att}}} \sum_{\substack{f \in F_{\text{acc}} \\ M_f \geq 4}} \frac{M_f - k_f}{M_f}.$$

$$A = \min\{1,\ C + 0.25H\} \qquad (1)$$

$$D = \frac{\sum_{f \in F_{\text{def}}} (M_f - k_f)}{\sum_{f \in F_{\text{def}}} M_f} \qquad (2)$$

Let G be the team-service-round cells retained after the challenge-wide first-infrastructure-error void rule, and let ρ_g be the effective credit assigned to cell g after missing-result, recovery, and `InternalError` adjudication.

$$R = \frac{\sum_{g \in G} \rho_g}{|G|} \qquad (3)$$

$$\text{Core} = 0.40A + 0.40D + 0.20\sqrt{AD} \qquad (4)$$

$$S = 100R\,\text{Core} \qquad (5)$$

Read Equations (1)-(5) in five steps:

1. `C` is the fraction of available opponent flags that team t captured.
2. `H` adds a bounded signal for captured flags that few eligible teams found. Equation (1) adds one quarter of `H` to `C` and caps offense `A` at `1`.
3. Equation (2) calculates defense `D` as the fraction of eligible attacker-flag pairs that remained uncaptured.
4. Equation (3) averages the effective checker credit ρ_g across retained service-round cells to produce SLA `R`.

5. Equation (4) combines offense and defense. Equation (5) multiplies the complete core by SLA and scales the answer to 100 points.

When a rate has a zero denominator, the implementation returns `0`. It clamps normalized rates and `Core` to `[0,1]`, clamps local points to `[0,100]`, and rejects negative or non-finite continuous evidence instead of producing an invalid score [1].

The geometric balance term rewards a team that both attacks and defends. SLA multiplies the complete local result: 50% SLA halves the service points, and 0% SLA reduces them to zero. rsctf scores each epoch independently and combines its service results into one 0-100 epoch result. Every complete epoch has equal weight in the official total.

Before play, organizers choose `n`, the number of rounds in a complete epoch, from the permitted range 1-64. This bound limits the amount of live evidence in one epoch. A shortened final epoch never receives full weight: `r_e` durable scoring rounds out of `n` configured rounds produce `q_e = r_e/n`. All expected checker work must also finish or be resolved before the epoch becomes final.

4.4 Challenge and epoch aggregation

Before scoring starts, organizers assign each challenge a service weight `w` from `0.8` to `1.2`. rsctf freezes this value with the `startRound` snapshot and reuses it for every epoch. The scorer divides each weight by the sum of all service weights, so all challenge contributions still fit within one 100-point epoch. Organizers can use the narrow range to represent a pre-event assessment of service difficulty or exposure to generic automated exploits. The weight does not change in response to live rarity or team performance.

$$c_i = \frac{S_i w_i}{\sum_j w_j} \qquad (6)$$

$$E = \sum_i c_i \qquad (7)$$

In Equation (6), S_i is the local 0-100 score for challenge i, and w_i is its frozen weight. Dividing by the sum of all weights produces contribution c_i. Equation (7) adds those contributions to epoch score E.

The epoch maximum therefore remains 100 regardless of the number of challenges. The scoreboard value for one challenge is its additive share of the total; it is not a separate score out of 100.

Let r_e be the number of stored scoring rounds currently represented in epoch e, where $1 \le r_e \le n$. Its weight is:

$$q_e = \frac{r_e}{n} \qquad (8)$$

A full epoch has $q_e = 1$ even while it remains Live. A current or final partial epoch has $q_e < 1$. For the Live score, Equation (9) includes every epoch that contains scoring rounds. For Settled, it includes only finalized epochs:

$$T = \frac{\sum_e q_e E_e}{\sum_e q_e} \qquad (9)$$

The platform averages epoch scores on a 0-100 scale instead of accumulating points. For a current total T with cumulative epoch weight W, a new epoch with score x and weight q changes the total by $\Delta T = q(x - T) / (W + q)$, where $|\Delta T| \le 100q / (W + q)$. A later complete epoch still has weight `1`, but one epoch moves an average less after the total already contains many epochs. This arithmetic does not prove that scores converge or that players remain engaged.

The first round with a complete registered team-service roster and prepared custom-checker files becomes the published `startRound`. Flags, captures, protected pairs, and checker credit from earlier rounds do not enter the ranked score [1].

4.5 SLA and infrastructure-fault adjudication

For SLA, rsctf builds a grid with one cell for every frozen service in every scoring round. Each cell contributes the effective credit shown below after the fault rules are applied.

Table 4. Checker outcomes and effective SLA credit.

Outcome	SLA treatment	Interpretation
`Ok`	`1.0`, except the next `Ok` whose most recent non-infrastructure verdict is `Mumble` or `Offline` receives `0.5`	The checker reports that the service is available, behaves correctly, and handles the current flag. Missing or `InternalError` samples between the failure and the next `Ok` do not remove the recovery rule.
`Mumble`	`0`	The service responds but violates the required behavior.
`Offline`	`0`	The service cannot be reached successfully.
Missing expected result	`0`	rsctf keeps the expected cell in the denominator instead of making a team's sample smaller.
`InternalError`	After `startRound`, carry the last resolved non-infrastructure credit; an isolated first error is `0`	A platform or checker fault creates no random bonus and does not repeatedly erase the last known service state.

rsctf voids a shared sample only when every frozen service for the same challenge and round receives its first `InternalError`. This pattern indicates a field-wide checker outage, so the scorer removes that challenge-round from every team's SLA denominator. The rule prevents one platform-wide failure from changing only selected team results [1].

The first `Ok` after `Mumble` or `Offline` earns `0.5`. For example, the sequence `Ok, Offline, Ok, Ok` receives credits `1.0, 0, 0.5, 1.0`. A crash and restart therefore cannot restore full SLA in the first healthy round. Event data would be required to determine whether this recovery penalty changes player behavior.

4.6 Bounded scarcity and pairwise defense

The rarity coefficient has two limits. Scarcity can add at most 25% of base capture coverage, and clamping prevents it from increasing offense `A` by more than 20 percentage points. One rare capture therefore cannot replace consistent attack coverage or create an unlimited jackpot.

In fields of at least three teams, pairwise `D` prevents one distinct capturer from erasing a whole flag. With 20 teams, one capturer removes one of 19 defense pairs, leaving `18/19 = 94.7%` of that flag's defense outcome. Ten capturers leave `9/19 = 47.4%`. With only two teams, `M=1`, so the sole opponent's capture does remove the flag's entire defense outcome. Pairwise accounting still records exactly which attacker-victim outcome changed.

Rarity and defense describe only what rsctf observed in accepted submissions. They do not reveal why a flag was or was not submitted. Organizers must therefore monitor deliberate withholding, flag sharing, coordinated cheating, and extra fake-team accounts, also called sybil accounts, throughout the event.

rsctf recalculates rarity from the eventual distinct capturer count `k`. It is not a permanent first-blood reward. An early Live scarcity bonus can shrink when more teams submit the same still-valid flag before the epoch settles.

5. ANALYTIC VERIFICATION AND SCOREBOARD INTERPRETATION

5.1 Worked scoring example

Accepted captures and scarcity determine offense `A`, protected pairs determine defense `D`, and checker credit determines SLA `R`. These rates produce service score `S`; the service scores then produce epoch score `E` and the Live and Settled totals.

Readers who do not need to reproduce the arithmetic can skip to the plain-language result after the equations. The calculation uses the same five steps listed in Section 4.3.

Consider a five-team event. Each team has `M=4` opponents. During one epoch, Team Blue's first service has eight attack opportunities and four accepted captures. The four captured flags were seen from `k=[1,2,2,3]` distinct capturers across the field. For accepted capture i, h_i denotes its normalized scarcity fraction $(M_i - k_i) \,/\, M_i$.

$$
\begin{aligned}
C &= \frac{4}{8} = 0.5000, \\
(h_1, h_2, h_3, h_4) &= \left(\frac{3}{4}, \frac{2}{4}, \frac{2}{4}, \frac{1}{4}\right) = (0.75, 0.50, 0.50, 0.25), \\
H &= \frac{0.75 + 0.50 + 0.50 + 0.25}{8} = 0.2500, \\
A &= \min\{1,\ 0.5000 + 0.25(0.2500)\} = 0.5625 .
\end{aligned}
$$

Two of Team Blue's own flags each received an `Ok`, flag-verified checker result and were therefore defense-eligible; they were captured by `k=[1,2]` opponents. No challenge-round was voided. The service accumulated `7.0` effective SLA credits across eight retained service-round cells:

$$
\begin{aligned}
D &= \frac{(4-1) + (4-2)}{4+4} = \frac{5}{8} = 0.6250, \\
R &= \frac{7}{8} = 0.8750, \\
\text{Core} &= 0.40(0.5625) + 0.40(0.6250) + 0.20\sqrt{(0.5625)(0.6250)} \approx 0.593585, \\
S &= 100(0.8750)(0.593585) \approx 51.9387 .
\end{aligned}
$$

The event has a second service scoring `70.00`. The first service weight is `1.2`; the second is `0.8`:

$$
\begin{aligned}
c_1 &= \frac{51.9387(1.2)}{2.0} \approx 31.1632, \\
c_2 &= \frac{70.0000(0.8)}{2.0} = 28.0000, \\
E &= c_1 + c_2 \approx 31.1632 + 28.0000 = 59.1632 .
\end{aligned}
$$

Suppose Team Blue has finalized epoch scores `59.1632` and `80.0000`. Settled is `69.58` after rounding. A current four-round epoch has two durable scoring rounds opened and currently scores `40.0000`, so its weight is `2/4 = 0.5`:

$$\begin{aligned} T_{\text{settled}} &= \frac{59.1632 + 80.0000}{2} = 69.5816 \approx 69.58, \\ q_{\text{live}} &= \frac{2}{4} = 0.5, \\ T_{\text{live}} &= \frac{59.1632 + 80.0000 + 0.5(40.0000)}{1 + 1 + 0.5} \\ &= 63.66528 \approx 63.6653 \approx 63.67\,. \end{aligned}$$

Team Blue's first service has 56.25% offense, 62.5% defense, and 87.5% SLA, producing 51.94 points. After service weighting, the epoch is 59.16; the two finalized epochs yield a 69.58 Settled score, while the weak open tail lowers the current Live projection to 63.67.

Live is lower than Settled here. That is valid: Live is the current weighted projection including unfinished evidence, not a promise that the final score will rise.

5.2 Balance-term sensitivity

The geometric term is positive only when both `A` and `D` are positive. Table 5 compares balanced and one-sided cases:

Table 5. Analytic sensitivity of the local service score.

Offense `A`	Defense `D`	SLA `R`	Local score	Reading
60%	60%	100%	60	Balanced performance retains the full 60.
90%	10%	100%	46	Strong attack cannot fully compensate for weak defense.
100%	0%	100%	40	Attack-only play still earns its base attack component, but no balance value.
60%	60%	75%	45	SLA scales the entire otherwise-60-point result.

5.3 Reading the scoreboard

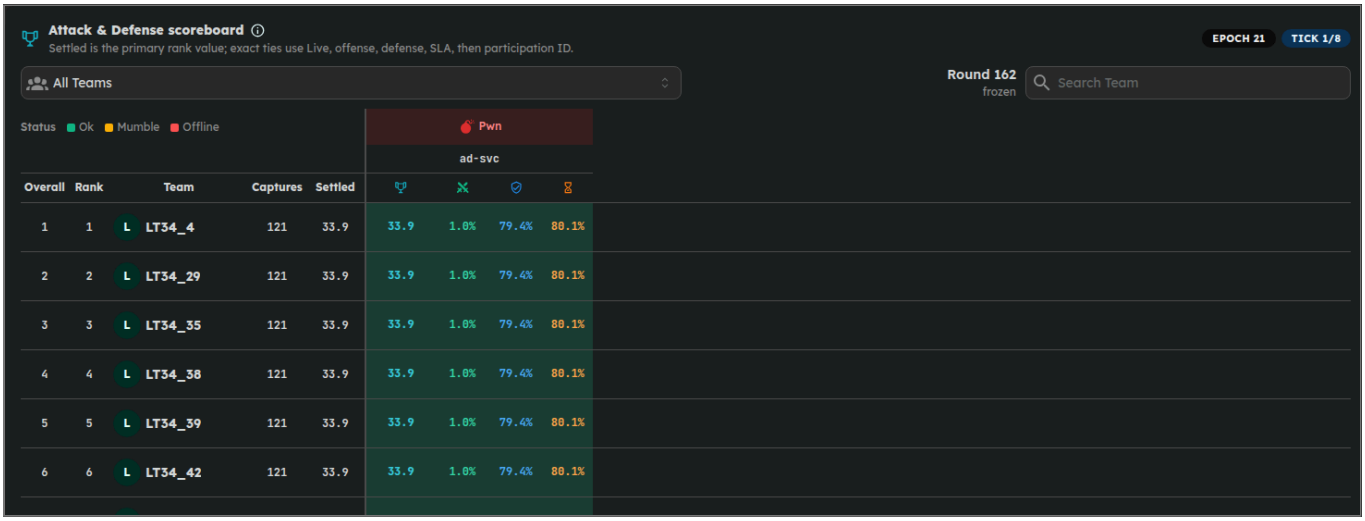

Figure 5. RSCTF EpochBalanced Attack & Defense scoring interface captured from the Docker Compose deployment on 14 July 2026 after the retained one-hour, 100-team event settled. The upper-right header reports Epoch 21 and Tick 1/8. The screen is application output, not a static mockup.

For a first reading, use Settled as the primary ranking value and Live for the current direction. When Settled is exactly equal, Live is the first tie-break; offense, defense, SLA, and participation ID follow. The component columns also help explain why the projection moved.

The deployed player board places official ranking and diagnostic rates in the same table. Figure 5 records the final Round 162 snapshot of the retained 100-team simulation. The visible page shows one service, ordinal ranks, 121 captures for each leading team, a Settled score of `33.9`, and the offense, defense, and SLA rates that produced that result.

- **Rank:** ordered by `Settled → Live → offense → defense → SLA → participation ID`. The participation ID provides a stable final order when every score and rate is equal.
- **Epoch and tick badges:** the upper-right header shows `Epoch N` and `Tick x/y`, identifying the current scoring group and the round's position within that group. The information button shows the configured tick length, epoch size, and current epoch round range.
- **Round countdown:** the current operational cycle and its intended end time.
- **Settled total:** weighted average of finalized epochs only.
- **Live total:** weighted average using all current evidence, including every not-yet-finalized full epoch and the open partial tail at weight `r/n`. It can rise or fall until settlement.
- **Per-challenge contribution:** the challenge's additive share after challenge and epoch weighting. Settled challenge contributions sum to Settled; projected challenge contributions sum to Live.
- **Offense, defense, and SLA rates:** projected diagnostics aggregated from current evidence. They explain direction but, after older rollups and nonlinear epoch scoring, cannot reconstruct the Settled contribution by themselves; they are not separate bonus pools.
- **Cell health color:** the most recent checker status, not a historical claim that every tick was healthy.
- **Open epoch:** provisional until its last flag lifetime closes. At game end, submissions close immediately; already-launched checks may finish before rsctf seals the final partial tail at the same fractional weight. A bounded grace records any still-missing result as `InternalError`; the carry, isolated-first-zero, or challenge-wide-void SLA policy then applies.
- **Finalized epoch:** settled evidence whose contribution no longer changes; after all epochs finalize, settled and projected totals match.

The list UI keeps detailed rows for the latest three epochs. Older rows are only hidden from that compact view; both totals always use the complete epoch history.

Organizers should retain accepted-flag, checker, and dispute telemetry for review.

5.3.1 Common scoreboard observations

Table 6. Interpretation of common Live and Settled observations.

Observation	Why it happens
The epoch's last round ended, but Settled did not move.	Flags issued near the end can still be submitted. The epoch remains Live until those windows and checker results close.
Live is below Settled.	A weak current partial epoch is included in Live at fractional weight but is not yet included in Settled.
A challenge local formula looks high, but its displayed contribution is lower.	The displayed value is normalized against all challenge weights so contributions add to one 0-100 team total.
A healthy service has no defense rate yet.	Defense eligibility needs an `Ok` result from a prepared flag-aware custom checker and qualified pair evidence.
A successful capture did not create a large rarity jump.	A flag submitted by few teams can contribute only a limited scarcity increment, subject to the `M >= 4` threshold and offense clamping.
The public board differs from a monitor view.	An organizer can freeze the player-facing view at the configured cutoff while authorized monitors continue to observe live evidence.

6. HUMAN-AI TEAM OPERATIONS

The formula scores repeated outcomes. It does not score team workflow, identify who made a tool, or measure how much work a human performed. The practices below reduce operational risk, but rsctf awards no direct points for following them. Teams can use AI to make work more repeatable, but they should test generated actions and require human approval before high-impact deployment changes.

6.1 Build three parallel workflows

Here, a **workflow** is a repeatable set of tools and team actions. Run these three workflows at the same time because an A&D round does not pause offense while defenders patch.

1. **Offense workflow:** poll current targets once per round, identify service versions, run bounded exploit jobs, parse flags strictly, deduplicate, and submit in small low-latency batches. Record victim, challenge, round, exploit path, and response for debugging.

2. **Defense workflow:** preserve a clean instance or image. Write patch scripts that are safe to run more than once, test the published checker contract, deploy one controlled change, observe at least one checker cycle, and keep an immediate rollback.
3. **Operations workflow:** monitor service process health, latency, disk, memory, flag handling, checker status, round transitions, API failures, and VPN reach. Page a human when automation cannot distinguish an infrastructure incident from a bad patch.

6.2 Treat AI output as untrusted input

- Require generated patches to pass functional and checker-contract tests.
- Review commands for destructive filesystem, firewall, credential, and process changes before they reach a live service.
- Prefer small diffs and reversible mitigations over broad rewrites during play.
- Test exploit parsers against malformed output so hallucinated text is never submitted as thousands of bogus flags.
- Rate-limit every loop and use target lists, not guessed address ranges.
- Keep a human approval path for resets, image changes, access-control changes, and any action that can affect all services.

6.3 Balance offense, defense, and SLA

- Consistent accepted coverage is more valuable than waiting only for a rare jackpot; rarity is deliberately bounded.
- A working patch is valuable only if the service still produces the required checker outcome; current rsctf does not automatically inject round flags into running managed containers.
- Continue attacking after a successful defense improvement; the balance term rewards teams that do both well.
- Continue defending after finding a strong exploit; offense and defense have equal base coefficients, while SLA multiplies both.
- Watch until submissions close. Old flags remain valid, open evidence can move Live, and later complete epochs still use `q=1`, although their marginal effect declines as cumulative epoch weight grows.

7. PLAYER READINESS CHECKLIST

7.1 Before scoring starts

- Connect the event VPN and confirm only expected routes use it.
- Register SSH access if enabled and store private keys safely.
- Verify every team service and challenge target from the toolkit.
- Run a baseline checker-equivalent smoke test before patching.
- Rotate, securely distribute, and store the team's shared A&D API token.
- Prepare target polling, exploit execution, deduplication, submission, logging, service monitoring, and rollback tooling.
- Agree on team roles and an escalation channel.

7.2 Every round

- Refresh the current round and target list.
- For BYOC, confirm the relay-updated shared flag file is visible to the service.
- For managed services, confirm that the runtime-updated flag file matches the current round before restarting or replacing the service process.
- Check each challenge's latest health result before and after patches.
- Run exploits against current opponent targets only.
- Submit valid captures promptly and process each per-flag status.
- Record unexpected checker, network, or scoreboard behavior with timestamps.

7.3 Before the event closes

- Keep exploit and submission workers active until the published deadline.
- Avoid high-risk last-minute patches without a tested rollback.
- Preserve logs and accepted-submission receipts for disputes.
- Wait for `fullySettled`; do not treat a remaining Live value as an award result.

8. STAKEHOLDER AND ORGANIZER GOVERNANCE

8.1 Precommit the scoring policy

Before the event, publish every setting that can change the score. An **immutable artifact identifier** is a value, such as a cryptographic hash, that lets organizers prove which checker file they used. Once scoring starts, rsctf locks the epoch and timing configuration, accepted-team roster, enabled challenge and checker configuration, and service weights. Organizers must also record the hash of each prepared checker and prevent checker files from being replaced during ranked play. If an organizer changes the end time or overrides scoring evidence, rsctf invalidates the affected stored epoch results and rebuilds them during the next scoreboard calculation.

Table 7. Precommitted event parameters, implementation ranges, and rationale.

Parameter	rsctf range/default	Governance purpose
Tick duration	`30-600` seconds; commonly `60`	Sets operational pace, checker load, and player reaction time.
Flag lifetime	`1-50` rounds; commonly `5`	Balances exploit/submission latency against stale-flag exposure.
Epoch size `n`	`1-64` rounds; default `8`	Controls settlement cadence and the rounds represented by a full-weight epoch.
Challenge weight	`0.8-1.2`	Allows a modest, preannounced service-difficulty adjustment.
Start, end, freeze	Event timestamps	Defines evidence acceptance, public visibility, and award closeout.
Scoring release	Version or commit identifier	Pins the formula and adjudication implementation used for the event.
Checker artifacts	SHA-256 digest per prepared checker	Detects replacement of checker code during ranked play.
Frozen roster and weights	Export at `startRound`	Reconstructs attack denominators, defense pairs, and challenge contributions.
Fault and closeout policy	Void rule and grace threshold	Precommits infrastructure-error treatment and final settlement timing.

The scoring start should remain automatic and singular. Confirm at least two accepted teams, the complete enabled service-row matrix for every accepted team, prepared custom-checker files, and actual service health before allowing `startRound` to lock. The first three conditions are automated; health validation is an organizer responsibility.

8.2 Checker validation

An organizer-supplied flag-aware checker must validate availability, protocol correctness, and the current flag without exposing secrets or creating a universal exploit. rsctf passes the flag to the checker and trusts an `Ok` exit; it does not independently inspect the checker's comparison logic. Test healthy, mumble, offline, recovery, timeout, and infrastructure-error paths under realistic load. A fallback TCP reachability probe is useful operationally but intentionally cannot create defense eligibility.

Monitor field-wide failures separately from team-local failures. The scorer can void a challenge-round only for the narrow all-service first-infrastructure-error case; other disputes need retained checker logs and organizer judgment.

8.3 Operational monitoring

Track at least:

- round creation delay and checker completion latency;
- BYOC flag-delivery failures, managed flag compatibility, target publication, and container churn;
- accepted, duplicate, expired, wrong, and self-attack submission rates;
- challenge-wide checker error correlation;
- service resets, shell access, unusual network paths, and control-plane load;
- per-flag capturer-count and submission-time distributions, together with independently corroborated indicators of flag sharing, collusion, or target-specific exceptions;
- public freeze behavior, rollup completion, and `fullySettled` status.

Outcome scoring does not replace competition-integrity monitoring. Maulana et al. separate hard evidence such as stolen flags from capped behavioral signals and zero-direct-score context signals in a two-stage CTF risk model; that distinction is useful when reviewing flag sharing, automation, and collusion without treating weak network correlations as direct guilt [6].

9. VALIDITY, FAIRNESS, AND LIMITATIONS

9.1 Enforced invariants

- A team score bounded to the interval 0-100 rather than an unbounded points race.
- Equal 40% base emphasis on offense and defense plus a 20% balance term.
- Local multiplicative SLA, so one healthy challenge cannot subsidize a broken one through an unrelated global availability bonus.
- A frozen start roster and shared attack opportunity denominators.
- Pairwise defense, so one capture removes one attacker-victim outcome.

- Bounded rarity with no small-field rarity below four opponents.
- Normalized, precommitted challenge weights.
- Duplicate and self-capture rejection.
- Equal complete epochs and fractional shortened tails.
- Explicit missing-check and field-wide infrastructure-outage treatment.
- Durable epoch rollups used by both totals even when older detail rows are omitted from the compact UI.

9.2 Threats to validity

Table 8. Scope of claims supported by the implemented evidence.

Claim	What the implementation establishes	Remaining limitation
Score is bounded	Formula and aggregation code clamp the team total to `0-100`.	A bounded score is not by itself a fair measure of skill.
Offense evidence exists	An authenticated valid target-bound flag was accepted.	It does not establish independent acquisition, operator identity, network path, or exploit provenance.
Pair remained protected	No accepted capture exists for that eligible attacker-flag pair.	It does not prove that the attacker tried or that a patch stopped the attempt.
Capture was scarce	Fewer frozen-roster teams submitted that same flag.	Capturer count does not prove novelty, independence, difficulty, or patch bypass.
Service earned checker credit	The configured checker returned an adjudicated result.	rsctf trusts organizer-supplied checker logic and cannot make a poor checker valid.
Equal epoch coefficients	Every complete epoch enters Equation (9) with weight `1`.	Comeback probability and player engagement still require event data; equal weighting proves only the coefficient rule.
Human or AI contribution	No direct evidence is recorded.	Outcome records cannot distinguish human work, AI assistance, or autonomous tooling.

Accepted captures and protected pairs are **proxies**: observable records used in place of direct measurements of exploit skill or patch effectiveness. This limits **construct validity**, meaning how well the score represents the skill it aims to measure. Evidence integrity also depends on correct checkers, secure tokens, reliable flag delivery, and accurate classification of infrastructure faults.

The repository contains no representative history of real competitive attacks. Its tracked simulator uses synthetic team profiles and omits delayed submissions across overlapping flag windows [7]. The report therefore has limited **external validity**: simulator results cannot establish how the model behaves across real events. Operational fairness also depends on comparable compute and network conditions, incident response, retained audit evidence, and enforcement against flag sharing, withholding, collusion, and target-specific exceptions.

Flag delivery has a specific implementation boundary. For BYOC, rsctf retries the relay stream within a bounded publication phase. A successful stream write shows that the relay accepted the payload, but no application-level acknowledgement proves that the defended service later read the shared file. A mid-round reconnect does not replay a phase that has already settled. For a running Docker- or Kubernetes-managed service, rsctf writes the current value to `RSCTF_FLAG_FILE` (`/flag` by default) through the container runtime and reads it back before checking the service. An image without `sh` or a writable flag path fails this verification. The round stores both the publication timestamp and the number of failed deliveries; checker adjudication then records the service outcome. The player-facing `flagsReady` field means that this bounded publication phase has settled, not that every offline participant acknowledged delivery.

rsctf scores accepted flags and checker results without requiring replay runners, patch declarations, or exploit uploads. Replay would require sandboxing untrusted code, while uploads would expose team artifacts; neither establishes human authorship. Because this design has not been compared experimentally with replay-based scoring, provenance claims remain an organizer adjudication task.

10. FINALIZATION, AWARDS, AND CORRECTIONS

At the published deadline, rsctf rejects new submissions and stops opening normal rounds. A checker pass that has already started can finish. If all expected results arrive, rsctf can seal the closeout immediately. Otherwise, a 240-second grace threshold makes missing results eligible for the `InternalError` fallback. A later supervisor pass performs the seal, so 240 seconds is an eligibility threshold rather than a strict upper limit. Ranked SLA then uses the same carry, isolated-first-zero, or challenge-wide-void policy described in Section 4.5 [1].

The final epoch keeps its real fractional weight. If an eight-round epoch ends after three durable rounds, its weight is `3/8`, not `1`. Game end closes flag acceptance, so the platform does not wait unused post-event lifetime rounds.

Awards, qualification exports, and public final statements should wait until the scoreboard reports `fullySettled`. At that point rollups cover the current epoch, the final visible round is sealed, and Live and Settled converge.

10.1 Disputes and corrections

Retain the following until the appeal window closes:

- round start/end rows and the published event configuration;
- frozen roster and challenge weight snapshot;
- issued flag identity metadata, without exposing live plaintext unnecessarily;
- accepted submission receipts and per-flag decisions;
- custom-checker outputs, effective SLA credit, and infrastructure incident logs;
- container reset, shell-access, VPN, and administrator action telemetry;
- scoreboard rollup generation and any administrative invalidation/rebuild record.

Finalized evidence no longer changes through ordinary play. For every administrative correction, record the actor, UTC timestamp, affected game, challenge, team, and round, original and replacement evidence, justification, invalidated rollup range, before-and-after totals, and public announcement identifier.

11. CONCLUSION

EpochBalanced converts repeated A&D outcomes into a score bounded from 0 to 100 while teams retain control of their exploits. Equations (1)-(5) combine accepted offense, observed pairwise defense, and service correctness; SLA scales the local result, and scarcity remains bounded. Equations (6)-(9) normalize challenge weights, assign coefficient `1` to every complete epoch, and give a shortened final epoch fractional weight. In an event with at least three teams, one distinct capturer removes only the corresponding attacker-victim pair. These are implementation guarantees. Claims about fairness, engagement, or human oversight require live-event data and a separate construct-validity analysis.

Accepted flags are durable competition outcomes, but they do not prove independent patch bypasses. An uncaptured pair records non-capture, not an attempted exploit. Organizers therefore need checker validation, equivalent infrastructure, anti-collusion rules, telemetry, and a documented appeal process.

Teams should attack, patch, measure, and retain rollback capacity until submissions close. Awards should use `fullySettled`; Live remains provisional.

APPENDIX A. FREQUENTLY ASKED QUESTIONS

A.1 Is a tick different from a round?

No. In this A&D implementation, tick and round name the same operational cycle. An epoch is different: it groups several scoring rounds.

A.2 Does a flag expire when its round ends?

No. It remains acceptable for the configured `L` rounds, from issue round `N` through `N+L-1`. The event deadline closes every remaining window.

A.3 Which epoch receives a late flag submission?

The epoch where the flag was issued. Submission time does not move evidence to a newer epoch.

A.4 Why is a complete epoch still Live?

Its rounds may be complete while its newest flags are still valid or checker results are still closing. Full length is not the same as finalized.

A.5 Can Live be lower than Settled?

Yes. Live includes unfinished evidence. A weak partial epoch can pull the current projection below the finalized-only average.

A.6 Does Live affect rank?

Yes, but only as a tie-breaker. The complete ordering is `Settled → Live → offense → defense → SLA → participation ID`.

A.7 Does rarity prove that we bypassed a patch?

No. It says fewer teams submitted the same accepted flag. That is a useful but bounded scarcity signal, not proof of exploit provenance or patch causality.

A.8 Does an uncaptured flag prove our patch worked?

No. It records an uncaptured eligible pair. The opponent may have failed, chosen another target, withheld a flag, or never attempted the exploit.

A.9 Why did an `Ok` checker earn half SLA?

The next `Ok` whose most recent non-infrastructure verdict is `Mumble` or `Offline` receives recovery credit `0.5`, even if missing or `InternalError` samples intervene. Infrastructure errors follow the separate policy above. Sustained healthy operation returns full credit.

A.10 Does rsctf run our exploit or require a patch declaration?

No. Your team manages and executes its own tooling. rsctf publishes targets and accepts captured flags; it does not need your exploit source or a patch claim.

APPENDIX B. NOMENCLATURE

Table 9. Symbols and scoreboard terms used throughout the paper.

Symbol or term	Definition
`N`	Round where a particular flag record was issued.
`L`	Configured flag lifetime in rounds.

Symbol or term	Definition
`n`	Configured number of scoring rounds in a complete epoch.
`r_e`	Durable scoring rounds currently represented in epoch `e`.
`N_att`	Frozen attack-opportunity count for one team-service epoch.
`F_acc`	Distinct opponent-flag opportunities captured by the scored team.
`F_def`	The scored team's own `Ok`, flag-verified defense-eligible flags.
`G`	Retained team-service-round cells after the field-wide void rule.
`rho_g`	Effective SLA credit assigned to retained cell `g`.
`M`	Frozen eligible opponent count for a flag, normally team count minus one.
`k`	Distinct accepted capturers of one flag.
`C`	Base accepted attack coverage.
`H`	Normalized bounded scarcity fraction for captures.
`A`	Final offense rate after bounded rarity and clamping.
`D`	Protected eligible opponent-pair fraction.
`R`	Local checker/SLA rate.
`S`	Local service score after SLA multiplication, bounded to `0-100`.
`w`	Precommitted challenge weight.
`c`	One challenge's normalized contribution to an epoch score.
`q_e`	Epoch weight `r_e/n`; it reaches `1` when all `n` rounds are represented.
`E`	One team's normalized 0-100 score for an epoch.
`T`	Weighted team total across epochs.
Live	Weighted total from all current evidence, finalized or not.
Settled	Weighted total from finalized epochs only.
Fully settled	Event closed, final round sealed, and every ranked epoch durably rolled up.
BYOC	Bring Your Own Container; the self-hosted mode in which a team runs its service container and relay agent on team-controlled infrastructure.

APPENDIX C. IMPLEMENTATION TRACEABILITY

Appendix C maps each implementation claim to its repository source. Paths are relative to the rsctf repository.

Table 10. Release-bound implementation traceability matrix.

Responsibility	Source of truth
Round creation, authoritative flag-record generation, scoring start gate	`src/services/ad/engine/rounds.rs`
Mode-specific flag publication and checker handoff	`src/services/cron/round_finish.rs`, `src/services/ad/engine/checker.rs`
Checker execution and status mapping	`src/services/ad/engine/checker.rs`

Responsibility	Source of truth
Round closeout and missing-result persistence	`src/services/ad/engine/persistence.rs`
Flag lifetime, duplicate/self checks, submission decisions	`src/controllers/game/ad/submit.rs`
Frozen roster and SQL evidence aggregation	`src/services/ad/scoring/evidence.rs`
Offense, rarity, defense, SLA, and local formula	`src/services/ad/scoring/formula.rs`
Challenge and epoch aggregation	`src/services/ad/scoring/aggregate.rs`
Durable epoch materialization	`src/services/ad/scoring/rollup.rs`
Live/Settled board construction and rank sorting	`src/services/ad/scoring/board.rs`
Player scoreboard presentation	`web/src/components/AdScoreboardTable.tsx`
Deterministic scoring sensitivity simulator	`tools/ad-scoring-sim/`

C.1 Core player-tooling HTTP surface

Table 11. Core player-facing A&D routes.

Method and route	Purpose
`GET /api/Game/{id}/Ad/State`	Current round and the caller team's service/flag state.
`GET /api/Game/{id}/Ad/Targets`	Current opponent targets, ports, and latest visible health.
`POST /api/Game/{id}/Ad/Submit`	Submit a batch of 1-100 captured flags.
`GET/POST/DELETE /api/Game/{id}/Ad/Token`	Read token hint/metadata, rotate it, or revoke it. POST returns new plaintext once; rotation replaces the team token.
`GET /api/Game/{id}/Ad/Vpn/Config`	Download the caller's WireGuard configuration.
`GET/POST/DELETE /api/Game/{id}/Ad/Ssh/Key`	Manage the caller's SSH public key when shell access is enabled.
`POST /api/Game/{id}/Ad/Ssh/Key/Generate`	Generate an SSH keypair when organizer policy allows it.
`POST /api/Game/{id}/Ad/Services/{serviceId}/Reset`	Reset an authorized team service.
`GET /api/Game/{id}/Ad/Services/{serviceId}/Snapshot`	Download an allowed post-game service snapshot.
`GET /api/Game/{id}/Ad/Byoc/Setup/{challengeId}`	Read the caller's BYOC setup material.
`GET /api/Game/{id}/Ad/Byoc/Compose/{challengeId}`	Generate the caller's BYOC Compose configuration.
`GET /api/Game/{id}/Ad/Scoreboard`	Read the ranked epoch board and per-challenge contributions.

The generated in-game toolkit is authoritative for the deployed host, game ID, authentication method, and enabled features. Do not hard-code example hostnames from this paper.

C.2 Reproducibility and data availability

The implementation audit was refreshed against the source tree shipped in the same release commit as this handbook on 14 July 2026. In a Git checkout, `git rev-parse HEAD` identifies that exact baseline. Repository artifacts include the formula unit tests, deterministic simulator, tracked output, and historical-audit query [1], [7]. Reproduce the software and document checks with:

```
cargo test services::ad::scoring
node tools/ad-scoring-sim/test.mjs
node tools/ad-scoring-sim/simulate.mjs --check
pnpm --dir docs pdf:ad
pnpm --dir docs build
```

The simulator's profiles, exploit diffusion, patch hazards, and availability are synthetic sensitivity inputs, not observations of real competitors. Its output can test arithmetic, boundedness, rank behavior, and failure modes; it cannot establish that the scoring policy measures skill or causes engagement.

APPENDIX D. DEPLOYMENT SECURITY AND FAIR PLAY

D.1 Bring Your Own Container (BYOC) challenges

In a BYOC challenge, your team runs the service and relay agent on its own machine. Follow the generated configuration from the toolkit. The downloaded setup script contains an image capability and creates a Compose file with the relay capability plus a WireGuard private key. Keep the script private, run it only on the dedicated BYOC host, and delete the downloaded copy after setup. The generated directory and credential files are owner-only. The optional BYOC SSH/admin shell requires deliberately uncommenting a Docker socket mount; leave it disabled unless that root-equivalent host access is acceptable on the dedicated worker.

> **Optional BYOC shell access**
>
> BYOC SSH can require mounting your host's Docker socket into the relay agent. This gives that agent root-equivalent control of your Docker host. Use a disposable, dedicated machine and opt out of shell access if you do not accept that risk.

Technical readers can consult the BYOC SSH internals.

D.2 Fair play

Do not attack the rsctf control plane, VPN hub, database, checker infrastructure, or another team's non-game systems. Do not flood submissions or availability checks. The target list defines the competition boundary.

For connection and player-side failures, see Health and troubleshooting.

REFERENCES

1. rsctf Project, “EpochBalanced scoring implementation,” repository-local source artifact, release commit containing this handbook, 14 July 2026.
2. M. Debono, “Announcing the Save CTFs Fund,” *OtterSec*, 7 July 2026. [Online]. Available: https://osec.io/blog/save-ctfs-fund/. Accessed: 11 July 2026.
3. FAU Security Team, “Rules,” *FAUST CTF 2025*, 2025. [Online]. Available: https://2025.faustctf.net/information/rules/. Accessed: 11 July 2026.
4. FAU Security Team, “CTF Gameserver architecture,” technical documentation, n.d. [Online]. Available: https://ctf-gameserver.org/architecture/. Accessed: 11 July 2026.
5. Shellphish, “How To Play iCTF,” *iCTF 2021*, 2021. [Online]. Available: https://ictf.cs.ucsb.edu/archive/ictf_2021/competition_website/howto.html. Accessed: 11 July 2026.
6. D. Maulana, I. W. C. Winetra, and I. N. R. W. Kesuma, “Cheating Detection in Capture the Flag Competitions Using Two-Stage Similarity Analysis and Tiered Weighted Risk Scoring,” *METHOMIKA: Jurnal Manajemen Informatika & Komputerisasi Akuntansi*, vol. 10, no. 1, pp. 366-372, Apr. 2026, doi: 10.46880/jmika.Vol10No1.pp366-372. [Online]. Available: publisher article page. Accessed: 11 July 2026.
7. rsctf Project, “A&D scoring simulator and deterministic report,” repository-local software and data artifact, schema version 5, commit `cb89a2654dfb808d62e9a10f3c0dfb71c55e5b66`, 11 July 2026.